

Aula 05 — Funções e procedimentos armazenados em PL/pgSQL

Disciplina: Banco de Dados II

Curso: Bacharelado em Sistemas de Informação

Semestre: 2026.2

Professor: Alysson M. Bruno

Data: 12/09/2026

Turma no Google Classroom: <https://classroom.google.com/c/ODY5MzE1NDkyODUx>

Objetivos

Ao concluir este estudo, você deverá conseguir:

- justificar por que uma regra de negócio pode ser armazenada dentro do SGBD;
 - reconhecer as partes de uma rotina em PL/pgSQL;
 - escrever funções em `LANGUAGE sql` e em `LANGUAGE plpgsql`;
 - declarar parâmetros, valores-padrão e variáveis;
 - devolver um valor simples ou um conjunto de linhas;
 - diferenciar função de procedimento e utilizar o comando `CALL`;
 - criar, substituir, inspecionar e remover rotinas no PostgreSQL.
-

1. Do comando SQL à regra de negócio

Nas Aulas 02 e 03 aprendemos a escrever comandos SQL. Na Aula 04, executamos o PostgreSQL em contêiner e observamos que o servidor é um programa independente das aplicações que o consultam. Agora daremos um passo além: em vez de apenas **enviar** comandos ao servidor, vamos **guardar comandos dentro dele**.

Considere uma loja com quatro tabelas: `cliente`, `produto`, `pedido` e `item_pedido`. Para saber quanto custa um pedido, escrevemos:

```
SELECT SUM(quantidade * preco_unitario)
FROM item_pedido
WHERE pedido_id = 1;
```

A consulta está correta. O problema aparece quando essa mesma conta precisa existir no site da loja, no aplicativo do vendedor e no relatório mensal da gerência. São três programas, escritos em linguagens diferentes, repetindo a mesma regra.

Suponha que a empresa passe a somar o frete ao total do pedido. Agora é preciso alterar os três programas. Se um deles for esquecido — o que costuma acontecer —, o sistema passará a informar dois

valores diferentes para o mesmo pedido. O erro não é de sintaxe: é de **consistência da regra**.

Ponto-chave: quando a mesma regra é escrita em vários lugares, a manutenção multiplica o risco de erro. Guardar a regra em um único ponto reduz esse risco.

A alternativa estudada nesta aula é escrever a regra **uma única vez, dentro do banco de dados**, e permitir que todas as aplicações a chamem pelo nome:

```
SELECT fn_total_pedido(1);
```

2. Programação no servidor de banco de dados

2.1 O que é uma rotina armazenada

Uma **rotina armazenada** (*stored routine*) é um trecho de código guardado dentro do banco de dados e executado pelo próprio servidor. O termo cobre dois objetos:

- **função** (*function*): sempre devolve um resultado e é chamada dentro de uma consulta;
- **procedimento** (*procedure*): executa uma ação e é chamado pelo comando CALL.

Além delas, existem os **gatilhos** (*triggers*), que disparam automaticamente diante de uma alteração nos dados. Os gatilhos são o tema da Aula 06 e dependem exatamente do que veremos aqui: um gatilho, no PostgreSQL, aciona uma função.

O conjunto formado por funções, procedimentos e gatilhos é o que se costuma chamar de **programação no lado do servidor** (*server-side programming*). Elmasri e Navathe (2019) apresentam esse recurso como a forma de aproximar o processamento dos dados, reduzindo o tráfego entre aplicação e servidor.

2.2 Vantagens

Vantagem	Explicação
Ponto único de manutenção	corrigir a rotina corrige o comportamento de todas as aplicações
Reúso	qualquer programa, em qualquer linguagem, chama a rotina pelo nome
Menor tráfego na rede	o cálculo ocorre onde os dados já estão, e só o resultado trafega
Padronização	o nome da rotina passa a ser o vocabulário comum da equipe
Segurança	é possível conceder permissão para executar a rotina sem conceder acesso direto às tabelas

O último item merece destaque. Uma rotina criada com a cláusula `SECURITY DEFINER` é executada com os privilégios de quem a criou, e não com os de quem a chama. Isso permite oferecer uma operação controlada a um usuário que não tem permissão sobre as tabelas envolvidas. O assunto será retomado na Aula 14, dedicada a segurança.

2.3 Limites e cuidados

Colocar código no banco não é sempre a melhor decisão. Observe:

- o código armazenado também precisa ser **versionado** em arquivos, junto com o restante do sistema;
- um defeito na rotina afeta **todas** as aplicações ao mesmo tempo, o que exige testes cuidadosos;
- rotinas escritas em PL/pgSQL não são portáteis para outro SGBD sem reescrita, pois cada fabricante tem a sua linguagem (Oracle usa PL/SQL; SQL Server usa T-SQL);
- regras de interface, navegação e apresentação **não** pertencem ao banco.

Critério prático: se a regra fala sobre **os dados** (como somar, validar, transformar, arquivar), ela costuma caber no banco. Se a regra fala sobre **a tela** ou sobre o fluxo do usuário, ela pertence à aplicação.

2.4 Um pouco de história

O padrão SQL incorporou a programação procedural na parte conhecida como **SQL/PSM** (*Persistent Stored Modules*), publicada em 1996. Cada fabricante implementou o conceito à sua maneira. No PostgreSQL, a linguagem procedural padrão é a **PL/pgSQL**, cujo nome significa *Procedural Language/PostgreSQL* e cuja sintaxe é fortemente inspirada na PL/SQL da Oracle.

O PostgreSQL admite ainda outras linguagens procedurais, como PL/Python, PL/Perl e PL/Tcl. Nesta disciplina usaremos apenas `sql` e `plpgsql`, que já vêm instaladas.

3. A linguagem PL/pgSQL

3.1 Estrutura de bloco

Todo código PL/pgSQL é organizado em **blocos**. A forma geral é:

```
[ DECLARE
    declarações de variáveis ]
BEGIN
    comandos
END;
```

A seção DECLARE é opcional: quando a rotina não precisa de variáveis, o bloco começa direto no BEGIN. O END encerra o bloco e é seguido de ponto e vírgula.

Atenção a uma armadilha de vocabulário: em PL/pgSQL, BEGIN **não** inicia uma transação. Ele apenas marca o início do trecho executável do bloco. O BEGIN que inicia transações é um comando SQL, estudado na Aula 07.

3.2 A delimitação por cifrões

Quando criamos uma rotina, o corpo dela é entregue ao servidor **como texto**. Historicamente esse texto era escrito entre aspas simples, o que obrigava a duplicar toda aspa interna. Para evitar isso, o PostgreSQL oferece a *dollar quoting*: o texto é delimitado por dois cifrões.

```
AS $$  
    -- aqui dentro podemos usar 'aspas simples' à vontade  
$$;
```

O delimitador pode receber um rótulo, útil quando há blocos aninhados:

```
AS $funcao$  
    ...  
$funcao$;
```

3.3 Comentários

```
-- comentário de uma linha  
  
/* comentário  
   de várias linhas */
```

4. Funções

4.1 O comando CREATE FUNCTION

A forma que usaremos nesta disciplina é:

```
CREATE OR REPLACE FUNCTION nome_da_funcao(lista_de_parametros)  
RETURNS tipo_do_resultado  
LANGUAGE linguagem  
AS $$  
    corpo  
$$;
```

Cada parte tem uma função específica:

Parte	Papel
CREATE OR REPLACE FUNCTION	cria a função ou substitui a versão existente
nome_da_funcao(...)	nome pelo qual a função será chamada e dados que ela recebe
RETURNS	tipo do valor devolvido
LANGUAGE	linguagem em que o corpo foi escrito
AS \$\$... \$\$	o corpo propriamente dito

A cláusula OR REPLACE é conveniente durante o desenvolvimento, pois evita apagar e recriar a função a cada correção. Ela não permite, porém, alterar o tipo de retorno nem os nomes dos parâmetros de saída: nesses casos é preciso remover a função antes.

4.2 Parâmetros

Cada parâmetro é declarado com **nome** e **tipo**:

```
CREATE OR REPLACE FUNCTION fn_total_pedido(p_pedido_id integer)
```

Adotaremos o prefixo p_ para parâmetros e v_ para variáveis. Essa convenção resolve, de forma preventiva, o erro mais comum do PL/pgSQL: dar a um parâmetro o mesmo nome de uma coluna. Quando isso ocorre, o servidor não sabe se pedido_id se refere à coluna da tabela ou ao parâmetro, e responde com a mensagem column reference "pedido_id" is ambiguous.

Um parâmetro pode ter **valor-padrão**:

```
p_percentual numeric DEFAULT 10
```

Assim, fn_desconto(120.00) e fn_desconto(120.00, 10) produzem o mesmo resultado. Parâmetros com valor-padrão devem vir **depois** dos que não têm.

Os parâmetros também têm **modo**:

Modo	Significado
IN	recebe um valor (é o padrão e pode ser omitido)
OUT	devolve um valor
INOUT	recebe e devolve
VARIADIC	recebe uma quantidade variável de valores

4.3 Variáveis e atribuição

As variáveis são declaradas na seção DECLARE e recebem valores com o operador :=.

```

DECLARE
    v_valor numeric;
    v_hoje date := CURRENT_DATE;    -- é possível já iniciar com um valor
BEGIN
    v_valor := p_preco * 0.9;

```

Também é possível declarar uma variável **com o tipo de uma coluna**, o que evita erros quando o esquema muda:

```

DECLARE
    v_preco produto.preco%TYPE;

```

4.4 Guardando o resultado de uma consulta

Para levar o resultado de uma consulta para dentro de uma variável, usamos `SELECT ... INTO`:

```

SELECT COALESCE(SUM(quantidade * preco_unitario), 0)
    INTO v_total
    FROM item_pedido
    WHERE pedido_id = p_pedido_id;

```

Três observações importantes:

1. a consulta deve devolver **uma linha**; se devolver várias, apenas a primeira é aproveitada;
2. se a consulta não devolver nenhuma linha, a variável fica com `NULL`;
3. `COALESCE(x, 0)` troca o `NULL` por zero — no exemplo, um pedido sem itens passa a valer zero em vez de “valor desconhecido”.

4.5 O que a função pode devolver

Declaração	Devolve
<code>RETURNS numeric</code>	um único valor numérico
<code>RETURNS text</code>	um único texto
<code>RETURNS boolean</code>	verdadeiro ou falso
<code>RETURNS void</code>	nada de útil
<code>RETURNS SETOF produto</code>	várias linhas com o formato da tabela <code>produto</code>
<code>RETURNS TABLE (col tipo, ...)</code>	várias linhas com colunas definidas na hora

Uma função que devolve conjunto é chamada **no FROM**, como se fosse uma tabela:

```

SELECT * FROM fn_pedidos_do_cliente(1);

```

Dentro da rotina, as linhas são produzidas por `RETURN QUERY`:

```

RETURN QUERY
  SELECT p.id, p.data_pedido, fn_total_pedido(p.id)
  FROM pedido p
  WHERE p.cliente_id = p_cliente_id
  ORDER BY p.id;

```

Atenção: os nomes declarados em RETURNS TABLE tornam-se variáveis dentro da função. Por isso, no exemplo acima, todas as colunas foram qualificadas com o apelido p. Sem essa qualificação, o servidor responde column reference "data_pedido" is ambiguous.

4.6 Funções em LANGUAGE sql

Quando o corpo da rotina é apenas uma consulta, não é necessário usar PL/pgSQL:

```

CREATE OR REPLACE FUNCTION fn_desconto_sql(
  p_preco      numeric,
  p_percentual numeric
)
RETURNS numeric
LANGUAGE sql
IMMUTABLE
AS $$
  SELECT round(p_preco - p_preco * p_percentual / 100, 2);
$$;

```

A comparação entre as duas linguagens ajuda a decidir:

Aspecto	LANGUAGE sql	LANGUAGE plpgsql
Corpo	uma consulta	um bloco de comandos
Variáveis	não possui	possui
Decisão e repetição	não possui	IF, CASE, LOOP, WHILE, FOR
Tratamento de exceções	não possui	possui
Otimização	pode ser embutida na consulta que a chamou	executada como bloco separado

A regra prática: comece pela versão mais simples e mude para PL/pgSQL quando precisar de lógica.

4.7 Classificação de volatilidade

Ao criar uma função, podemos informar ao planejador de consultas como ela se comporta:

Classificação	Significado
IMMUTABLE	com os mesmos argumentos, devolve sempre o mesmo resultado e não lê o banco
STABLE	não altera o banco e devolve o mesmo resultado dentro de uma mesma consulta
VOLATILE	pode alterar o banco ou variar a cada chamada (é o padrão)

Declarar corretamente ajuda o otimizador a evitar recálculos. `fn_desconto_sql` é `IMMUTABLE` porque depende apenas dos seus argumentos; `fn_total_pedido` seria `STABLE`, porque lê tabelas. Esse tema volta nas Aulas 10 e 11, sobre planos de execução.

5. Procedimentos

5.1 Criação e chamada

Procedimentos existem no PostgreSQL desde a versão 11. A criação é semelhante à da função, mas **sem** a cláusula `RETURNS`:


```

CREATE OR REPLACE PROCEDURE sp_baixar_estoque(
    p_produto_id integer,
    p_quantidade integer
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_estoque integer;
BEGIN
    SELECT estoque
    INTO v_estoque
    FROM produto
    WHERE id = p_produto_id;

    IF v_estoque IS NULL THEN
        RAISE EXCEPTION 'Produto % não encontrado.', p_produto_id;
    END IF;

    IF v_estoque < p_quantidade THEN
        RAISE EXCEPTION 'Estoque insuficiente: há % unidades e foram pedidas %.',
            v_estoque, p_quantidade;
    END IF;

    UPDATE produto
    SET estoque = estoque - p_quantidade
    WHERE id = p_produto_id;

    COMMIT;
END;
$$;

```

A chamada usa o comando CALL:

```
CALL sp_baixar_estoque(3, 1);
```

Não existe `SELECT sp_baixar_estoque(...)`. Tentar chamar um procedimento como se fosse função produz a mensagem `sp_baixar_estoque(integer, integer) is a procedure seguida da orientação para usar CALL`.

Os comandos `IF` e `RAISE EXCEPTION` aparecem aqui apenas para tornar o exemplo realista; ambos serão estudados em detalhe na Aula 06.

5.2 Controle de transação

Esta é a diferença mais importante entre os dois objetos. Uma **função** é avaliada dentro do comando que a chamou; ela participa da transação em curso, mas não pode encerrá-la. Um **procedimento** é chamado por um comando próprio e, por isso, pode executar `COMMIT` e `ROLLBACK` no seu corpo.

Esse recurso é útil em rotinas de manutenção que processam muitos registros: o procedimento confirma o trabalho em blocos, em vez de manter uma transação gigantesca aberta.

Restrição importante: o COMMIT dentro do procedimento só funciona quando o CALL **não** está dentro de uma transação aberta pelo usuário. Se você digitar BEGIN; antes do CALL, o servidor recusa a operação com a mensagem `invalid transaction termination`. No `psql`, que confirma cada comando automaticamente, o exemplo desta aula funciona como está.

5.3 Devolvendo valores por parâmetro

Um procedimento pode devolver informação por meio de parâmetros INOUT:

```
CREATE OR REPLACE PROCEDURE sp_abrir_pedido(  
    p_cliente_id    integer,  
    INOUT p_pedido_id integer  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO pedido (cliente_id)  
    VALUES (p_cliente_id)  
    RETURNING id INTO p_pedido_id;  
END;  
$$;  
  
CALL sp_abrir_pedido(2, NULL);
```

O CALL devolve uma linha com o valor final do parâmetro — no caso, o identificador do pedido recém-criado.

5.4 Quadro-resumo

Critério	Função	Procedimento
Devolve resultado	sempre	não, salvo por INOUT
Como é chamada	dentro de SELECT, WHERE, FROM	com CALL
Controla transação	não	sim
Pode ser usada em gatilho	sim	não
Uso típico	calcular, transformar, consultar	executar, alterar, manter

6. Manutenção das rotinas

6.1 Listar e ler

No psql:

```
\df          -- lista funções e procedimentos
\df fn_*      -- lista apenas o que começa com fn_
\sfn fn_total_pedido -- mostra o código-fonte da rotina
```

Fora do psql, a mesma informação está no catálogo:

```
SELECT routine_name, routine_type
FROM   information_schema.routines
WHERE  routine_schema = 'public'
ORDER BY routine_name;
```

6.2 Substituir e remover

```
DROP FUNCTION fn_desconto_sql(numeric, numeric);
DROP PROCEDURE sp_baixar_estoque(integer, integer);
```

Informar os **tipos dos parâmetros** é necessário porque o PostgreSQL admite **sobrecarga**: duas rotinas podem ter o mesmo nome, desde que recebam parâmetros diferentes. Sem os tipos, o servidor não sabe qual delas remover — a menos que exista apenas uma com aquele nome.

6.3 Onde o código fica guardado

O código-fonte das rotinas é armazenado no catálogo do sistema, na tabela `pg_proc`. Isso significa que a rotina acompanha o banco: ela é copiada pelo `pg_dump` (Aula 13) e restaurada junto com as tabelas. Ainda assim, mantenha os arquivos `.sql` versionados junto com o código da aplicação.

7. Hands-on: construindo as rotinas passo a passo

Esta seção reproduz o laboratório do encontro. O roteiro completo, com o checklist, é o **roteiro de laboratório** publicado junto com esta apostila no Google Classroom; os scripts já vêm prontos na máquina do playground, em `~/lab05`.

7.1 Preparar o ambiente

Abra <https://labs.iximiuz.com/playgrounds/BD2-AULA05-dedf2f1b> e clique em **Start Playground**. É o ambiente preparado para esta aula: a imagem do PostgreSQL 17 já vem baixada e os scripts do laboratório já estão em `~/lab05`. Na aba **Term**, execute:

```
podman run -d --name bd2-pg \  
-e POSTGRES_PASSWORD=bd2 \  
-e POSTGRES_DB=loja \  
docker.io/library/postgres:17
```

Verifique:

```
podman ps  
podman exec bd2-pg pg_isready -U postgres
```

Resultado esperado: o contêiner aparece ativo e o pg_isready responde accepting connections.

Entre no cliente de linha de comando:

```
podman exec -it bd2-pg psql -U postgres -d loja
```

Resultado esperado: o prompt passa a ser loja=#.

A senha bd2 é didática e vale apenas para um ambiente de estudo. Nunca reutilize senhas de exemplo em sistemas reais.

Uma sessão de playground dura algumas horas. Ao encerrá-la, a plataforma oferece duas opções: **Stop**, que guarda o estado do ambiente — arquivos e contêineres — para você retomar o trabalho depois, e **Destroy**, que apaga tudo definitivamente. Para continuar o laboratório em casa, escolha **Stop**. Ainda assim, anote os comandos e os resultados importantes: eles são o seu material de revisão.

7.2 Criar o esquema da loja

O arquivo 01_esquema.sql já está na máquina e cria as quatro tabelas com os dados de exemplo. Saia do psql com \q e execute:

```
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab05/01_esquema.sql
```

Volte ao psql com podman exec -it bd2-pg psql -U postgres -d loja e confira:

```
\dt  
SELECT * FROM produto ORDER BY id;
```

Os pedidos de exemplo têm os seguintes totais:

Pedido	Itens	Total
1	2 teclados e 1 cabo HDMI	275,00
2	1 monitor	900,00
3	3 mouses	240,00

7.3 Primeira função: LANGUAGE sql

```
CREATE OR REPLACE FUNCTION fn_desconto_sql(
    p_preco      numeric,
    p_percentual numeric
)
RETURNS numeric
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT round(p_preco - p_preco * p_percentual / 100, 2);
$$;
```

Teste:

```
SELECT fn_desconto_sql(120.00, 10);
```

Resultado esperado: 108.00.

Agora aplique a função a todas as linhas de uma tabela:

```
SELECT nome, preco, fn_desconto_sql(preco, 15) AS promocao
FROM produto
ORDER BY id;
```

A função é avaliada **linha a linha**, exatamente como uma expressão comum.

7.4 A mesma regra em PL/pgSQL

```
CREATE OR REPLACE FUNCTION fn_desconto(  
    p_preco      numeric,  
    p_percentual numeric DEFAULT 10  
)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_valor numeric;  
BEGIN  
    v_valor := p_preco - p_preco * p_percentual / 100;  
    RETURN round(v_valor, 2);  
END;  
$$;
```

Teste as duas formas de chamada:

```
SELECT fn_desconto(120.00, 25);  
SELECT fn_desconto(120.00);
```

Resultado esperado: 90.00 e 108.00. Na segunda chamada, o parâmetro omitido assumiu o valor-padrão.

7.5 Função que consulta o banco

```
CREATE OR REPLACE FUNCTION fn_total_pedido(p_pedido_id integer)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_total numeric;  
BEGIN  
    SELECT COALESCE(SUM(quantidade * preco_unitario), 0)  
        INTO v_total  
        FROM item_pedido  
        WHERE pedido_id = p_pedido_id;  
  
    RETURN v_total;  
END;  
$$;
```

Teste:

```
SELECT fn_total_pedido(1);

SELECT id, fn_total_pedido(id) AS total
FROM pedido
ORDER BY id;
```

Resultado esperado:

```
id | total
----+-----
 1 | 275.00
 2 | 900.00
 3 | 240.00
```

Compare com a tabela da seção 7.2: os valores conferem.

7.6 Função que devolve uma tabela

```
CREATE OR REPLACE FUNCTION fn_pedidos_do_cliente(p_cliente_id integer)
RETURNS TABLE (
    pedido_id integer,
    data_pedido date,
    valor_total numeric
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT p.id, p.data_pedido, fn_total_pedido(p.id)
        FROM pedido p
        WHERE p.cliente_id = p_cliente_id
        ORDER BY p.id;
END;
$$;
```

Teste:

```
SELECT * FROM fn_pedidos_do_cliente(1);
```

Resultado esperado: duas linhas, referentes aos pedidos 1 e 2, com os totais 275.00 e 900.00.

Experimento sugerido: retire os apelidos p. do SELECT interno, recrie a função e chame-a novamente. O servidor responderá que a referência de coluna é ambígua. Reponha os apelidos e observe o erro desaparecer. Provocar o erro de propósito é a melhor forma de reconhecê-lo depois.

7.7 Procedimento

Digite o procedimento sp_baixar_estoque apresentado na seção 5.1 e teste:

```
SELECT id, nome, estoque FROM produto WHERE id = 3;  
CALL sp_baixar_estoque(3, 1);  
SELECT id, nome, estoque FROM produto WHERE id = 3;
```

Resultado esperado: o estoque do monitor cai de 4 para 3.

Agora provoque a falha:

```
CALL sp_baixar_estoque(3, 500);
```

Resultado esperado:

```
ERROR:  Estoque insuficiente: há 3 unidades e foram pedidas 500.
```

Confira que o estoque permaneceu em 3. A rotina interrompeu a execução antes do UPDATE, e a regra passou a valer para qualquer aplicação conectada ao banco — inclusive para quem usar o psql diretamente.

7.8 Inspecionar e limpar

```
\df  
\sf fn_total_pedido  
DROP FUNCTION fn_desconto_sql(numeric, numeric);  
\df
```

Ao final, saia com \q e, se desejar apenas desligar o banco, execute `podman stop bd2-pg`. Ao fechar o playground, lembre-se da diferença entre **Stop** e **Destroy** descrita na seção 7.1.

8. Erros comuns e como interpretá-los

Mensagem	Causa provável	Correção
no language specified	falta a cláusula LANGUAGE	acrescente LANGUAGE plpgsql ou LANGUAGE sql
unterminated dollar-quoted string	o \$\$; final não foi digitado	feche o corpo da rotina
column reference "x" is ambiguous	variável ou parâmetro com nome de coluna	use os prefixos p_ e v_ e qualifique as colunas
function fn_x(integer) does not exist	nome errado ou tipo diferente do declarado	confira com \df
sp_x(...) is a procedure	procedimento chamado com SELECT	use CALL
query has no destination for result data	SELECT sem INTO dentro de PL/pgSQL	acrescente INTO ou use PERFORM
control reached end of function without RETURN	falta RETURN em algum caminho	garanta um RETURN em todas as saídas

Leia sempre a mensagem completa: além do texto, o PostgreSQL informa o contexto (CONTEXT: PL/pgSQL function ... line N), que aponta a linha exata do problema dentro do corpo da rotina.

9. Estudo orientado pelo celular

Se você não tiver computador disponível, é possível estudar assim:

1. leia os slides na ordem, do Passo 0 ao Passo 11;
2. copie em um caderno o esqueleto de uma função e escreva ao lado o nome de cada parte;
3. para cada rotina do laboratório, escreva com suas palavras: o que ela recebe, o que ela faz e o que ela devolve;
4. explique a diferença entre SELECT fn_total_pedido(1) e CALL sp_baixar_estoque(3, 1);
5. confira os totais dos pedidos manualmente e compare com os resultados da seção 7.5;
6. responda ao questionário no Google Classroom.

10. Atividade de consolidação

Parte A — leitura de código

Explique, em uma frase cada, o que fazem: DECLARE, :=, INTO, RETURN e CALL.

Parte B — escrita

Escreva uma função fn_estoque_baixo(p_limite integer) que devolva, em forma de tabela, o id, o nome e o estoque dos produtos cujo estoque esteja abaixo do limite informado. Use RETURNS TABLE e

qualifique as colunas.

Parte C — análise

Um colega escreveu uma **função** com COMMIT no corpo e recebeu um erro. Explique por que isso acontece e qual objeto ele deveria ter usado.

Parte D — questões objetivas

Responda às dez questões do arquivo `exercícios-aula05.md` no Google Classroom.

11. Síntese

- Uma **rotina armazenada** guarda, dentro do banco, uma regra que antes se repetia em cada aplicação.
 - Toda rotina é formada por **nome, parâmetros e corpo**; o corpo é delimitado por cifrões.
 - LANGUAGE `sql` resolve cálculos simples; LANGUAGE `plpgsql` acrescenta variáveis, decisões e repetições.
 - Uma **função** sempre devolve algo e é chamada dentro de consultas; pode devolver um valor ou uma tabela.
 - Um **procedimento** executa uma ação, é chamado com CALL e pode confirmar ou desfazer a transação.
 - As rotinas são objetos do banco: podem ser listadas, lidas, substituídas e removidas.
 - Na próxima aula, essas rotinas passarão a ser acionadas automaticamente pelos **gatilhos**.
-

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2019.

IXIMIUZ LABS. **Playgrounds**. 2026. Disponível em: <https://labs.iximiuz.com/playgrounds>. Acesso em: 10 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — Chapter 43: PL/pgSQL — SQL Procedural Language**. 2026. Disponível em: <https://www.postgresql.org/docs/17/plpgsql.html>. Acesso em: 10 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — CREATE FUNCTION**. 2026. Disponível em: <https://www.postgresql.org/docs/17/sql-createfunction.html>. Acesso em: 10 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — CREATE PROCEDURE**. 2026. Disponível em: <https://www.postgresql.org/docs/17/sql-createprocedure.html>.

Acesso em: 10 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18.6, 17.11, 16.15, 15.19 e 14.24 released**. 2026. Disponível em: <https://www.postgresql.org/about/news/>. Acesso em: 10 set. 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020. E-book. Disponível em: Minha Biblioteca.